Deep Learning Practices (DLP)

Assignment 03 — Object Detection, Semantic Segmentation and Recurrent Networks

Three compulsory questions · 100 marks · individual submission

Everything you need is in this folder: datasets are downloaded by a script, starter code is provided for every question, and a report template is included. Read sections 1–3 before you write any code.

1. Aim and learning outcomes

This assignment covers the two dense vision tasks — object detection and semantic segmentation — and then sequence modelling with recurrent networks. You are not asked to implement any architecture from scratch: the detectors and the segmentation networks are loaded pretrained, and the recurrence in Question 3 is PyTorch's own nn.RNN / nn.LSTM / nn.GRU. The marks are for experimental design, measurement, failure analysis and application judgement. All three questions are compulsory.

- **LO1 —** fine-tune and fairly compare a two-stage detector (Faster R-CNN) against a one-stage detector (YOLOv8), and explain the mechanisms that produce the difference.
- **LO2 —** fine-tune an encoder–decoder segmentation network and evaluate it with region-overlap metrics rather than pixel accuracy.
- **LO3 —** train recurrent models on a real sequence task, measure the vanishing-gradient behaviour that separates an LSTM from a vanilla RNN, and say what follows for practice.
- **LO4 —** map each model family onto real applications, with constraints (latency, object size, error cost) argued from your own measurements.
- **LO5 —** report experiments honestly: controlled comparisons, correct metrics, stated compute, and diagnosed failure cases.

Question	Topic	Dataset	Marks
Q1	Object detection: R-CNN family vs YOLO	PASCAL VOC 2007, 5-class subset	35
Q2	Semantic segmentation	Oxford-IIIT Pet trimaps	35
Q3	Recurrent networks: RNN vs LSTM	IMDB movie reviews (25k/25k)	30

2. Ground rules

- **No architectures from scratch.** Detectors and segmentation networks come pretrained from torchvision, Ultralytics and segmentation-models-pytorch. The recurrent layers come from torch.nn. You will not hand-code an RPN, a decoder or an LSTM gate, and no marks are available for doing so.
- **Use the starter code.** starter/ contains a working script per question. Each has one or two blocks marked TODO(student) — short, conceptual edits (which head to replace, which layers to freeze, which loss to use, how to pack a padded sequence, how to pool it). In total they are well under 40 lines of code. You may change anything else in the scripts; say so if you do.
- **Allowed libraries:** PyTorch, torchvision, Ultralytics, segmentation-models-pytorch, torchmetrics, pycocotools, OpenCV, scikit-learn, matplotlib, pandas. For Question 3 you may NOT use a pretrained language model, a transformer, or pretrained word vectors — the embedding is trained with the model.

- **Fair comparison is marked.** Within a question, keep the seed, data split and training budget identical across the runs you compare, and change one thing at a time. Both Q1 detectors must use the same split; the Q3 cells must use the same hidden size and epoch count.
- **Report only what you ran.** Every number in your report must be traceable to a file under runs/. Quoting a figure from a paper or a blog post as your own result is treated as misconduct.
- **Compute.** Q1 and Q2 need a GPU — a single free Colab T4 session is enough. Q3 runs on a laptop CPU in well under half an hour per configuration, so a GPU outage cannot cost you the whole assignment.
- **Seeds.** The seed is 42 in config.yaml. One run per configuration is acceptable; do not claim statistical significance from a single run.

3. Setup and suggested timeline

Run these once, in order. The three datasets come to about 1.8 GB.

```
pip install -r requirements.txt
python env_check.py                          # verifies torch, GPU, every library
python setup/download_datasets.py --all     # VOC 2007 + Oxford-IIIT Pet + IMDB
python setup/voc_to_yolo.py                  # the shared 5-class detection split
```

Day	Work	Rough GPU time
1	Setup and Q1: YOLOv8 zero-shot and fine-tuned, then Faster R-CNN, then the head-to-head table	60–105 min
2	Q2: baseline U-Net, frozen-encoder ablation, no-pretraining control, loss study, DeepLabV3	90–150 min
3	Q3: three cells, the length study, the gradient-flow figures — then make_tables.py and the write-up	30–60 min (or CPU)

Start the long vision runs before you write up the previous question — the GPU and you can work in parallel. Question 3 is the cheapest, so leave it last only if your GPU access is reliable; if it is not, do Q3 first.

4. Question 1 — Object detection: the R-CNN family vs YOLO (35 marks)

Dataset: PASCAL VOC 2007 restricted to five classes — person, car, dog, cat, bicycle — with the split built once by setup/voc_to_yolo.py. Both detectors must use that identical split; do not regenerate it between runs. Models: YOLOv8n (COCO-pretrained, Ultralytics) and Faster R-CNN ResNet-50-FPN (COCO-pretrained, torchvision). Scripts: starter/q1a_yolo.py, starter/q1b_faster_rcnn.py, starter/make_tables.py.

The two detectors represent the two families that have defined the field: region-proposal, two-stage detectors descended from R-CNN, and dense, single-pass detectors descended from YOLO. This question is about measuring that difference honestly and then deciding which one you would actually deploy.

Task	What to do	Marks
1.1	Dataset report: images per split, objects per class, and one sentence stating what the five numbers on a YOLO label line mean. Comment on the class imbalance you will have to interpret later.	4
1.2	Zero-shot baseline: python starter/q1a_yolo.py --zero-shot. Report mAP@0.5 with no fine-tuning, and explain both why it is already far above zero and why it is not a clean measure of zero-shot ability on our label set.	4

1.3	Fine-tune YOLOv8n. Report mAP@0.5, mAP@0.5:0.95, per-class AP, precision and recall, training time and milliseconds per image. Include the PR curve and confusion matrix that Ultralytics writes to the run folder.	7
1.4	Fine-tune Faster R-CNN: complete the TODO(student) head replacement, then train. Report the same metrics, plus mAP for small vs large objects. In writing: what are the two outputs of the box predictor you replaced, and why did the RPN not need replacing?	8
1.5	Head-to-head table on the same device: mAP@0.5, mAP@0.5:0.95, per-class AP, parameters, model size (MB), latency, FPS and training minutes. Add two qualitative figures — one image where YOLOv8 is clearly better, one where Faster R-CNN is — and say what distinguishes them.	7
1.6	Mechanism and applications (max 500 words). Mechanism: the RPN and ROI align of a two-stage detector against a dense anchor-free head; how each assigns ground truth to predictions; the role of NMS and the confidence threshold. Applications: pick TWO of autonomous driving, retail shelf auditing, medical imaging, industrial inspection, wildlife monitoring or video surveillance; state each one's real constraints (frame rate, object size, cost of a false negative, hardware budget) and say which detector you would deploy for each, arguing from your own numbers in 1.5.	5

Deliverables: the head-to-head comparison table, the per-class AP table, two qualitative figures; at most 900 words of analysis.

5. Question 2 — Semantic segmentation (35 marks)

Dataset: Oxford-IIIT Pet trimaps — every pixel is labelled pet, background or boundary. Models: U-Net with an ImageNet-pretrained ResNet-34 encoder (segmentation-models-pytorch) and DeepLabV3-ResNet50 with COCO weights (torchvision). Script: starter/q2_segmentation.py.

Detection draws boxes; segmentation commits to every pixel. The boundary class here is thin and heavily outnumbered, which is what makes the metric choice and the loss choice matter.

Task	What to do	Marks
2.1	Data sanity: show four image/mask overlays, and report the class pixel frequency (pet / background / boundary) measured over at least 200 training masks. State what that imbalance implies for your metric choice, and explain in one or two sentences why masks must be resized with nearest-neighbour interpolation.	4
2.2	Baseline: fine-tune U-Net with the ImageNet encoder. Report mIoU, mean Dice, pixel accuracy, per-class IoU and training time, with training curves and the qualitative grid the script saves.	8
2.3	Complete TODO(student) 1 and run --freeze-encoder. Compare mIoU, trainable parameters and training time against task 2.2, then say when you would prefer the frozen version in practice.	6
2.4	Pretraining control: --encoder-weights none. Quantify how much of your final mIoU comes from the pretrained encoder rather than from the architecture.	4
2.5	Loss study: complete TODO(student) 2 and run --use-dice. Report per-class IoU for cross-entropy and for cross-entropy + Dice, and explain the effect on the boundary class specifically.	6
2.6	Architecture, failures and applications. Run DeepLabV3-ResNet50 once at the same image size and epoch count; compare mIoU, parameter count and inference latency. Pick two failure images and diagnose each. Then name one application where the boundary class is the part that matters (for example surgical margins in medical imaging, drivable-area edges in autonomous driving, or field boundaries in satellite mapping) and say which of your runs you would ship for it, and why.	7

Pixel accuracy is not an acceptable headline metric for this dataset — explain why, and argue from mIoU and per-class IoU instead.

Deliverables: one results table with at least five rows; curves and a qualitative grid per model; at most 700 words of analysis.

6. Question 3 — Recurrent networks: RNNs and LSTMs (30 marks)

Dataset: the IMDB Large Movie Review Dataset — 25,000 labelled training reviews and 25,000 test reviews, binary sentiment. Models: torch.nn.RNN, torch.nn.LSTM and torch.nn.GRU behind one wrapper, with an embedding trained from scratch. Script: starter/q3_rnn_lstm.py. No pretrained language model, transformer or pretrained word vectors.

A vanilla RNN and an LSTM differ in one structural detail — the gated cell state that carries information along an additive path — and this question asks you to measure what that detail is worth, rather than assert it.

Task	What to do	Marks
3.1	Data and representation: vocabulary size, out-of-vocabulary rate, and the review-length distribution (mean, median, 90th percentile, fraction truncated) — the starter prints all of these. Explain why the batch carries a `lengths` vector alongside the padded token tensor, and what goes wrong in a 'last hidden state' classifier if padding is fed to the recurrence.	4
3.2	Complete both TODO(student) blocks (packing, and the mean-pooling readout), then train RNN, LSTM and GRU at the same hidden size, epochs and learning rate. Report test accuracy, total and recurrent-layer parameter counts, and training time in one table. Also report the packed-vs-unpacked difference for at least one cell, and the 'last' vs 'mean' readout for your best cell.	8
3.3	Sequence-length study: re-run RNN and LSTM at --max-len 50, 200 and 500 and plot accuracy against length for both. Which model degrades, at what length, and why would you expect that from the architecture?	7
3.4	Gradient flow: run both cells with --grad-flow and plot the per-timestep gradient norms for RNN and LSTM on one logarithmic axis. Quote the ratio between the first and last timestep for each. Connect this figure explicitly to your 3.3 result, and say what the gradient norms printed during training (and the effect of --no-clip) tell you about exploding, as opposed to vanishing, gradients.	6
3.5	Applications and error analysis (max 400 words). Pick TWO sequence applications — for example sentiment or intent classification, machine translation, speech recognition, handwriting recognition, clinical time series, demand forecasting or anomaly detection in sensor streams — and for each state the sequence length involved, whether it needs the whole sequence or a streaming decision, and which cell you would use and why. Then show two reviews your best model gets wrong and say what about them is hard (negation, a late reversal, sarcasm, a very long review).	5

An LSTM has roughly four times the recurrent parameters of an RNN at the same hidden size. Report both counts and state which comparison you chose — matched hidden size or matched parameters. Either is defensible; not saying which is not.

Deliverables: the cell-comparison table, the length-vs-accuracy plot, the gradient-flow figure, two error cases; at most 700 words of analysis.

7. Report requirements

- Ten to sixteen pages, submitted as PDF. Start from report/Report_Template.docx; it already has the required sections.
- Number and caption every figure and table, and refer to each one in the text. A figure nobody discusses earns nothing.
- Every number must be reproducible from a file under runs/. Generate your tables with python starter/make_tables.py rather than retyping numbers.
- Include a compute statement: device, approximate total GPU time, and library versions (paste the env_check.py output).
- State the random seed, and finish each question with two or three sentences on what you would try with more compute.
- Write in your own words. Quoting the assignment brief back at length is not analysis.

8. Submission

Submit one archive named <student_id>_DLP_A1.zip, at most 50 MB, containing:

```
report.pdf                  your report
code/                       your modified starter scripts + config.yaml
report/tables.md, *.csv     generated by starter/make_tables.py
runs/                       results.json and figures for all three questions
                            (DELETE every *.pt checkpoint)
AI_use_declaration.md       see section 10
```

Do not submit the datasets or model checkpoints.

9. Marking rubric

The marks listed per task above are allocated according to these criteria.

Criterion	Share	A strong answer
Implementation and correctness	35%	Runs complete, the TODO blocks are implemented correctly, metrics are computed on the right split, no label, padding or normalisation bugs.
Experimental control	20%	One variable changed at a time; seed, split and training budget held fixed across compared runs; identical data for both Q1 detectors and matched capacity in Q3.
Analysis and diagnosis	25%	Explains why the numbers came out that way, names concrete failure modes from your own figures, and distinguishes a model limitation from a data problem.
Applications judgement	10%	Real constraints for named domains, and a model choice argued from your own measurements rather than from folklore.
Report quality and reproducibility	10%	Clear tables and captioned figures, stated compute and versions, numbers traceable to runs/, within the page limit.

Marks are lost for: an uncontrolled comparison presented as a conclusion; pixel accuracy quoted as the headline segmentation metric; latency numbers measured on different devices in one table; comparing recurrent cells at

different hidden sizes without saying so; figures with no axis labels; and any number that cannot be traced to a run.

10. Academic integrity and AI use

- You may use AI assistants and online resources for debugging, explanation and plotting. You must declare them in AI_use_declaration.md: which tool, for what, and which parts of the submission it touched.
- You remain responsible for every number and every claim in the report. You may be asked to re-run any experiment and to explain any line of code you submitted.
- Sharing code or results with another student, or submitting runs you did not perform, is misconduct and is handled under the university policy.

11. Datasets used

Dataset	Size	Labels	Used in
PASCAL VOC 2007	~10k images, ~900 MB	20 object classes with bounding boxes; we use 5	Q1
Oxford-IIIT Pet	7,349 images, ~800 MB	per-pixel trimap: pet / background / boundary	Q2
IMDB movie reviews	50k documents, ~130 MB	binary sentiment, 25k train / 25k test	Q3
COCO / ImageNet	not downloaded	source of the pretrained weights you fine-tune in Q1 and Q2	Q1, Q2

All three are standard academic benchmarks fetched by the setup script; there is nothing to register for and nothing to scrape. Cite them in your report.

12. Compute notes and troubleshooting

- **Colab:** Runtime → Change runtime type → T4 GPU. Cache data/ in Google Drive so a disconnect does not cost you the download again.
- **CUDA out of memory:** halve batch_size in config.yaml, then reduce img_size. DeepLabV3 at 256 px needs roughly 6 GB at batch size 8.
- **Windows DataLoader stalls:** set num_workers=0 in the starter scripts.
- **VOC or IMDB mirror fails:** see the troubleshooting section of README.md — both have a documented manual-download fallback.
- **First Ultralytics run:** downloads yolov8n.pt (about 6 MB) automatically.
- **Q3 accuracy stuck at 0.50:** that is chance. Check the learning rate (a vanilla RNN often needs 5e-4), check that clipping is on, and check that your readout is not reading padded positions — which is what TODO 1 fixes.
- **Everything is slower than stated:** check env_check.py reports CUDA available = True. If it is False you are training on CPU, which is fine for Q3 and painful for Q1 and Q2.